

Daily Report cron確認・解析手順まとめ

対象: /home/pi/gmail_sorter/daily_report_mail.py の17時自動実行確認

結論

今回の確認では、crontabの設定、cronサービス、5分ごとの受付処理、Daily Reportメール送信ログのいずれも正常でした。したがって「実行されていない」のではなく、メール確認側で見落としていた可能性が高い、という判断になりました。

確認項目	結果	判断
crontab設定	17時の daily_report_mail.py 実行行あり	正常
cronサービス	active (running)	正常
5分ごとの受付処理	import_online_to_excel.py のCRON記録あり	稼働中
Daily Reportログ	「Daily Reportを送信しました。」が複数回あり	送信成功
/var/log/syslog	存在しない	OS環境差。異常ではない

1. crontab の設定確認

まず、Pi3で crontab の内容を確認します。

```
crontab -l
```

確認された主な設定は次の通りです。

```
* /5 * * * * /home/pi/gmail_sorter/venv/bin/python /home/pi/gmail_sorter/import_online_to_excel.py >>
/home/pi/gmail_sorter/gmail_sorter.log 2>&1
#0 17 * * * /home/pi/gmail_sorter/venv/bin/python /home/pi/gmail_sorter/daily_report.py >>
/home/pi/gmail_sorter/daily_report.log 2>&1
0 17 * * * /home/pi/gmail_sorter/venv/bin/python /home/pi/gmail_sorter/daily_report_mail.py >>
/home/pi/gmail_sorter/daily_report_mail.log 2>&1
```

17時にメール版Daily Reportを送る行は、次の1行です。

```
0 17 * * * /home/pi/gmail_sorter/venv/bin/python /home/pi/gmail_sorter/daily_report_mail.py >>
/home/pi/gmail_sorter/daily_report_mail.log 2>&1
```

この意味は、毎日17:00に venv 内の Python で daily_report_mail.py を実行し、標準出力とエラーを daily_report_mail.log に記録する、ということです。

2. cronサービスの稼働確認

cron自体が動いているかを確認します。

```
systemctl status cron
```

結果に次が出ていれば、cronサービスは正常です。

Active: active (running)

今回の確認では、cron.service は enabled かつ active running でした。

3. 5分ごとの受付処理が動いているか確認

cronの状態表示内に、次のような記録がありました。

```
CRON ... import_online_to_excel.py
```

これは、5分ごとの受付処理 `import_online_to_excel.py` が実際に呼び出されていることを示します。したがって、`cron`の基本動作は正常と判断できます。

4. Daily Reportメールのログ確認

Daily Reportメール用ログを確認します。

```
tail -n 80 /home/pi/gmail_sorter/daily_report_mail.log
```

今回のログには、次の成功メッセージが複数回ありました。

Daily Reportを送信しました。
送信先: allshimanto@gmail.com

このため、`daily_report_mail.py` は実行されており、メール送信処理も成功していると判断しました。

5. syslogが無い場合の確認方法

`cron`の実行記録を確認するため、次のコマンドを試しました。

```
grep CRON /var/log/syslog | tail -n 30
```

しかし、今回の環境では次の結果でした。

```
grep: /var/log/syslog: No such file or directory
```

これはOS環境による違いで、異常とは限りません。この場合は `journalctl` を使います。

```
journalctl -u cron
```

17時前後だけを見る場合は、次のように指定します。

```
journalctl -u cron --since "today 16:50" --until "today 17:10"
```

6. Gmail側の確認

今回のようにログでは送信成功しているのにメールが見当たらない場合、Gmail側で見落としている可能性があります。次の検索語で確認します。

from:allshimanto@gmail.com Daily Report

または、次でも探せます。

東京四万十会 Daily Report

確認場所は、受信トレイ、すべてのメール、迷惑メール、プロモーション、送信済みです。送信元と送信先が同じ `allshimanto@gmail.com` のため、通常の受信メールより見つけにくい場合があります。

7. 今後の標準確認手順

17時レポートが見当たらない場合は、以下の順に確認します。

```
# 1. Gmailで検索
from:allshimanto@gmail.com Daily Report

# 2. Pi側でDaily Reportログ確認
tail -n 80 /home/pi/gmail_sorter/daily_report_mail.log

# 3. cronサービス確認
systemctl status cron

# 4. crontab設定確認
crontab -l

# 5. 必要ならcron実行記録確認
journalctl -u cron --since "today 16:50" --until "today 17:10"
```

8. 必要に応じた改善案

現在の daily_report_mail.log は成功メッセージだけで、実行日時が分かりにくい状態です。今後、ログに日時を明示したい場合は、cron行を次のように変更します。

```
0 17 * * * echo "=== $(date '+ %Y- %m- %d %H: %M: %S') daily_report_mail start ===" >>
/home/pi/gmail_sorter/daily_report_mail.log 2>&1; /home/pi/gmail_sorter/venv/bin/python
/home/pi/gmail_sorter/daily_report_mail.py >> /home/pi/gmail_sorter/daily_report_mail.log 2>&1
```

**cron内では % をそのまま書くと特別扱いされるため、 \ %
のようにエスケープします。すぐに必要な変更ではありませんが、運用確認を楽にする改善です。**

最終まとめ

今回の解析で、17時Daily Reportの仕組みは正常に動いていることが確認できました。今後、メールが見当たらない場合でも、Pi側のログを見ることで「送信済みかどうか」を判断できます。